WORD LADDER GAME

Project Plan & Phased Development Roadmap

Temporary project name • Monorepo • Android-first MVP

Purpose
Build the game in small, testable phases. Start with a polished offline Android single-player experience. Add backend services and web administration only after core gameplay is reliable.

1. Product Vision

Create a fun, approachable, and strategically engaging word-transformation puzzle game. Players move from a starting word to a target word by changing exactly one letter per move. Every submitted word must be valid for the selected level and follow the game rules.

Core gameplay example

CASE → BASE → BAKE → CAKE → TAKE

This is an illustrative route only. The game must calculate the actual shortest path for each level; examples must not be treated as proof of optimality.

Product principles

- Easy to understand in seconds, with room for planning and optimization.
- Fair rules: clearly distinguish invalid attempts from valid moves.
- Reward efficient solutions while allowing alternative valid routes when level design permits them.
- Offline-first MVP: no account, network connection, or backend required to play.
- Build maintainable code and keep mobile, web, and backend projects separated by folder.

2. Repository Structure

Use the existing GitHub repository as a monorepo. Keep each application in its own folder:

Path	Purpose
mobile/	Native Android player app; build this first.
web/	Admin dashboard for levels, dictionary/content, and analytics; later phase.
backend/	API, account services, online progression, leaderboards, and multiplayer; later phase.
docs/	Product decisions, game rules, API contracts, and release notes.

Important: do not generate or scaffold web/ and backend/ during Phase 1 unless explicitly requested. The initial release should remain small and focused.

3. Phased Roadmap

Phase 1 — Offline Android MVP (START HERE)

Goal: a playable, polished single-player game on a real Android phone.

- Create the native Android app in mobile/ using Kotlin and Jetpack Compose (unless the current Google AI Studio workflow requires a different supported structure; explain deviations first).
- Create home, level-selection, gameplay, level-complete, and simple settings/help screens.
- Include predefined, hand-authored levels with start word, target word, allowed move limit, and expected optimal path length.
- Allow both tapping letter tiles and typing a complete candidate word.
- Validate that each accepted move keeps the same word length, changes exactly one letter, and exists in the approved dictionary.
- Clearly distinguish invalid input from a valid move. Invalid words must not advance the level.
- Track current word, move count, remaining moves, hints, completion, and star rating.
- Give bonus rewards for optimal solutions. Document scoring; do not unfairly reject valid alternative routes just because they differ from one preselected route.

- Use a small offline dictionary appropriate to the selected level lengths. Document source and licence; do not assume third-party dictionary data can be redistributed.
- Calculate or verify shortest paths for all shipped levels with graph search or trusted precomputed data. Test each optimal path length.
- Save progress locally so it survives app restarts.
- Use readable text, clear selected-letter states, accessible touch targets, understandable error feedback, and layouts that work on different phone sizes.
- Test on a physical Android phone using USB debugging. An emulator is optional.
- Add README setup/build/test instructions and commit working Phase 1 code to the existing repository.

Phase 1 acceptance criteria

- User can open the app, choose a level, play it, and reach the target word.
- Every accepted move changes exactly one letter and is a valid dictionary word.
- Invalid input gives understandable feedback and is not counted as a successful move.
- Move limits, hints, optimal-solution bonus, and stars behave consistently.
- Progress remains saved after app restart.
- All levels have verified shortest-path/optimal-length data.
- Project builds and runs on a physical Android phone.

Phase 2 — Content and Game-Feel Polish

Goal: make the core loop enjoyable and add replay value.

- Expand the campaign with curated difficulty progression.
- Improve animations, optional sound/vibration, transitions, and feedback without distracting from play.
- Add daily challenge using bundled/date-selected challenges; keep it offline if no backend exists.
- Add hint types and a help/tutorial flow explaining the one-letter rule.
- Add local statistics: levels completed, optimal solves, streaks, and best results.
- Add automated tests for validation, path calculation, scoring, and persistence.

Phase 2 acceptance criteria

- Difficulty increases predictably and all new levels pass validation.
- Players can understand rules without external instructions.
- Core logic has repeatable automated tests.

Phase 3 — Backend Foundation

Goal: introduce online features without coupling core gameplay to the network.

- Create backend/ as an ASP.NET Core Web API when online features are ready.
- Choose and document database, hosting, authentication, logging, and deployment before implementation.
- Define versioned API contracts for accounts, level/content delivery, progress, daily challenges, and leaderboards.
- Validate competitive scores server-side; do not trust client-reported optimality or rewards.
- Keep offline play working without network access.

Phase 3 acceptance criteria

- API contracts and error responses are documented.
- Core endpoints have automated tests and basic security protections.
- App can still launch and play offline.

Phase 4 — Web Admin Dashboard

Goal: manage game content without releasing a new mobile app for every content change.

- Create web/ as a separate admin application.
- Support authorized management of levels, start/target words, move limits, hints, and publication status.
- Validate content before publishing: word lengths, dictionary membership, and shortest-path metadata.
- Add audit history and preview/test a level before publishing.
- Add usage/content analytics only when reliable backend data exists.

Phase 4 acceptance criteria

- Only authorized admins can change published content.
- Invalid levels cannot be published.
- Published levels can be fetched by supported app versions.

Phase 5 — Accounts, Leaderboards, and Social Competition

Goal: introduce persistent identity and fair competition.

- Add optional accounts and cloud progress sync.
- Add global/friends leaderboards with server-side result validation and basic anti-cheat controls.
- Add daily rankings, streaks, trophies, and player profiles.
- Plan privacy, account deletion, data retention, moderation, and abuse reporting before social features launch.

Phase 5 acceptance criteria

- Competitive scores are validated server-side.
- Ranking and rewards are understandable to players.
- Privacy and account lifecycle flows are documented and tested.

Phase 6 — PvP and World/Theme Progression

Goal: expand only after single-player and online foundations are stable.

- Design turn-based or real-time PvP after deciding the match format.
- Add themed worlds/regions, unlockable content, seasonal events, and advanced progression.
- Measure performance, retention, matchmaking quality, and operating costs before scaling.

Phase 6 acceptance criteria

- Match rules, disconnect handling, and tie-breakers are documented.
- Multiplayer is tested for fairness, reliability, and abuse cases.

4. Phase 1 Technical Guidance

Suggested architecture

- Native Android UI: Kotlin + Jetpack Compose.
- Keep game rules and shortest-path logic separate from UI composables.
- Use a repository/data layer for level definitions, dictionary lookup, and local progress.
- Use reliable local persistence suitable for the chosen project template.
- Keep level data in versioned JSON or Kotlin data structures for the MVP.
- Use deterministic tests for validation, pathfinding, scoring, and persistence.

Core rules to implement explicitly

- All words in a ladder must have the same length.
- A move is valid only if exactly one character changes and the resulting word is in the approved dictionary.
- Start and target words must be valid dictionary entries.
- Every level must have at least one path from start to target.
- Optimal path length must be computed or independently verified; never claim a path is shortest without verification.
- Define whether the move counter counts only valid moves or all submissions. Recommended: count accepted valid moves and show invalid attempts separately if useful.
- Define whether the move limit counts moves/edges or words/steps; use the same definition in level data, UI, scoring, and tests.
- Hints must specify whether they reveal a next word, a letter, or an optimal route.

5. Instructions for Google AI Studio — Phase 1 Only

Upload this plan to Google AI Studio and use the prompt below. Keep the first generation scoped to Phase 1. Do not build the future backend, admin dashboard, accounts, or multiplayer now.

```
You are helping implement Phase 1 of the attached Word Ladder Game project plan. The GitHub
repository already exists and is a monorepo. First inspect the repository/current project structure
and explain what is present before changing files. Implement ONLY the offline native Android MVP in
mobile/. Do not build web/ or backend/ yet. Prefer Kotlin + Jetpack Compose if supported by the
current workflow. If you cannot create/edit the repository directly, provide a complete project
export and explain how to place it into mobile/ without overwriting unrelated files. Implement a
playable app with home, level selection, gameplay, completion, and help/settings screens;
predefined levels; tile tapping and full-word typing; dictionary validation; exact-one-letter move
validation; move limit; hints; optimal-solution bonus and stars; and local progress persistence.
Keep game logic separate from UI. Include automated tests for move validation,
shortest-path/optimal-length verification, scoring, and progress logic. Verify every shipped level
has a valid path and correct optimal path length. Do not claim a path is shortest unless
algorithm/tests verify it. Give clear invalid-input feedback and do not count invalid input as a
successful move. Document move-counting and scoring rules. Use dictionary data that can legally be
included and document its source/licence. Do not add accounts, cloud sync, ads, analytics SDKs,
PvP, leaderboards, payments, or backend dependencies in Phase 1. The app must work offline. Before
large changes, propose a short implementation plan and list assumptions. Then implement
incrementally, run available tests/build checks, fix errors, and summarize files changed, how to
run/build, how to test on a physical Android phone, and limitations. Do not invent successful
build/test results; state what you actually verified.
```

6. Development Workflow and Git Practices

- Keep main branch in a buildable state; use short-lived feature branches for larger changes.
- Make small commits after working milestones: scaffold app, move validation, path solver, progress persistence.
- Review generated code instead of accepting large unverified changes blindly.
- Run tests and build checks after meaningful changes.
- Never commit passwords, signing keys, API secrets, or machine-specific files.
- Update README.md whenever setup, build, test, or repository structure changes.

7. Scope Boundaries and Decisions to Revisit

- Final public game name and branding: decide later.
- Exact UI theme, typography, icon, and sound design: use a simple temporary MVP style.
- Dictionary size and word-length range: choose based on level set and licence suitability.
- Database, hosting, authentication provider, and deployment: decide before Phase 3.
- PvP format and ranking rules: decide before Phase 5/6.
- Monetization: out of scope until the core game proves enjoyable.

8. Definition of Done for Phase 1

- Project opens and builds with documented steps.
- App installs and runs on a physical Android phone.
- A complete level can be played from start to target.
- Move validation and shortest-path facts are covered by tests.
- Local progress persists across app restarts.
- No backend or internet connection is required for core gameplay.
- README includes setup, build, test, and device-install instructions.
- Changes are committed to the existing GitHub repository.